

Servicio Nacional de Aprendizaje SENA

Centro de Diseño y Metrología

Análisis y Desarrollo de Software

Ficha 3171608

Taller Sentencias Iniciales MongoDB

Santiago Muñeton Hernandez

Instructora Angelica Triana

06 de abril de 2026

Bogotá D.C.

Insertar Documentos

1. Abrir la base de datos

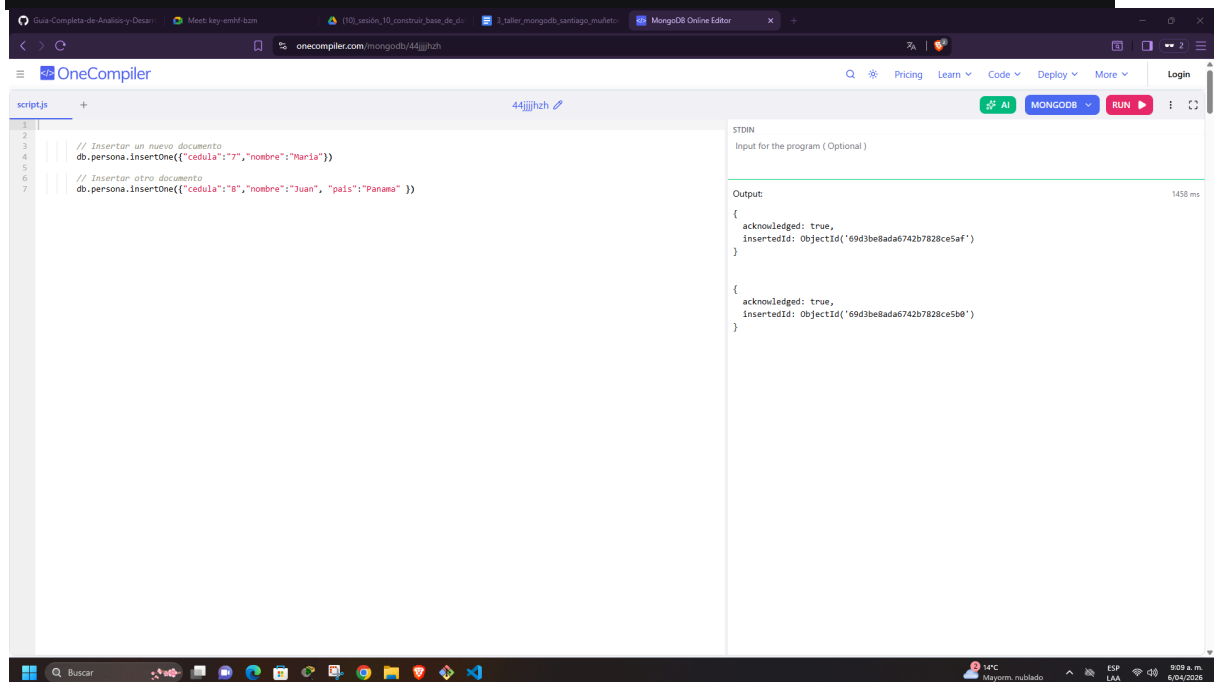
```
use("santiagoencodigo_mongodb")
```

2. Insertar un nuevo documento

```
db.persona.insertOne({"cedula":"7","nombre":"Maria"})
```

3. Insertar otro documento

```
db.persona.insertOne({"cedula":"8","nombre":"Juan", "pais":"Panama" })
```



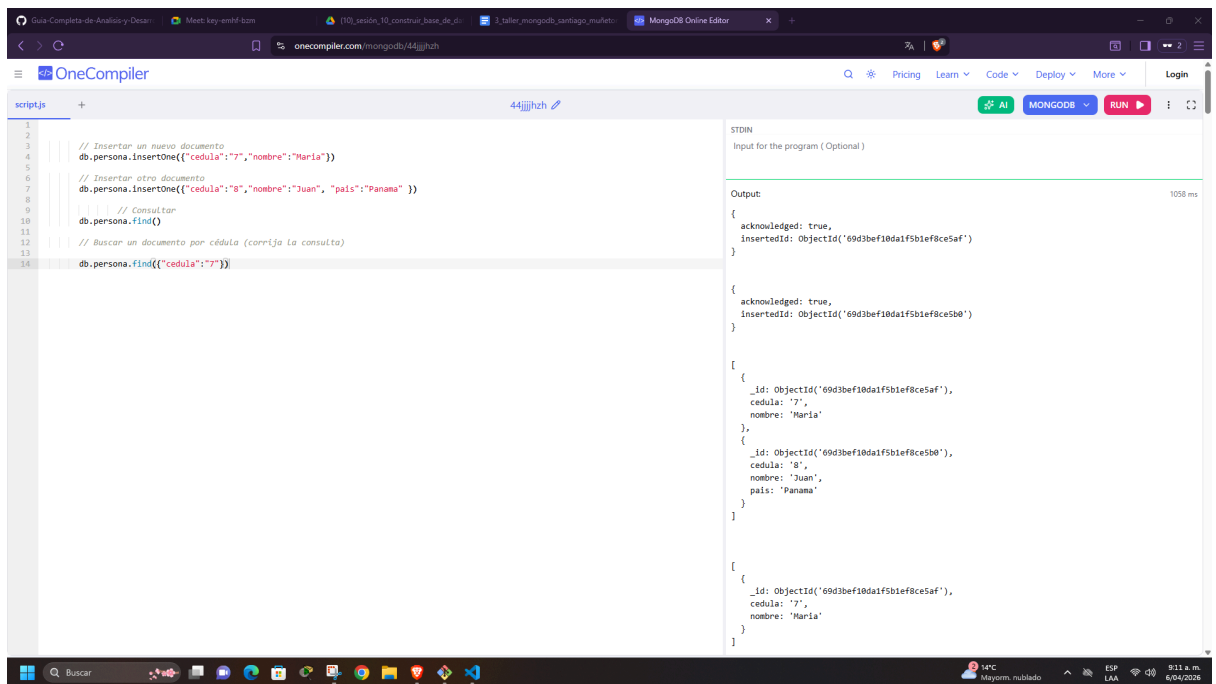
Consulta de Documentos

4. Consultar

```
db.persona.find()
```

5. Buscar un documento por cédula (corrija la consulta)

```
db.persona.find({"cedula":"7"})
```



Documento embebido

6. Agregar un documento embebido con la información del padre y la madre

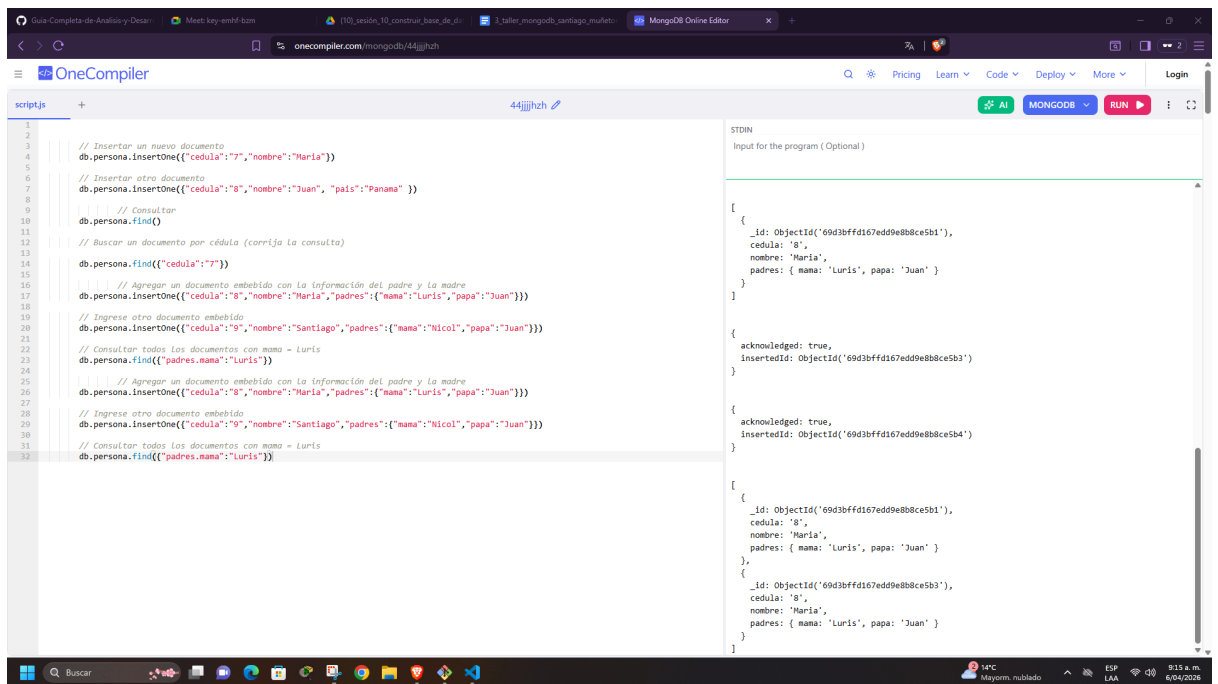
```
db.persona.insertOne({"cedula":"8","nombre":"Maria","padres":{"mama":"Luris","papa":"Juan"}})
```

7. Ingrese otro documento embebido

```
db.persona.insertOne({"cedula":"9","nombre":"Santiago","padres":{"mama":"Nicol","papa":"Juan"}})
```

8. Consultar todos los documentos con mama = Luris

```
db.persona.find({"padres.mama":"Luris"})
```



Conteste: ¿Cómo es la sintaxis para buscar en un documento incrustado o embebido?

Para buscar documentos incrustados o embebidos en MongoDB. La sintaxis consiste en escribir “db.” después el nombre de la colección “coleccion”, después el método “.find” seguido del nombre del campo “campoPrincipal” y seguido de un “.” el nombre del subcampo, después “:” y el nombre del valor que se quiere buscar.

Entonces por ejemplo:

```
db.persona.find({"padres.mama":"Luris"})
```

```
[
  {
    _id: ObjectId('69d3c36573174c5bcb8ce5b1'),
    cedula: '8',
    nombre: 'Maria',
    padres: { mama: 'Luris', papa: 'Juan' }
  },
  {
    _id: ObjectId('69d3c36573174c5bcb8ce5b3'),
    cedula: '8',
    nombre: 'Maria',
    padres: { mama: 'Luris', papa: 'Juan' }
  }
]
```

Eliminar Documentos

9. Eliminar un documento

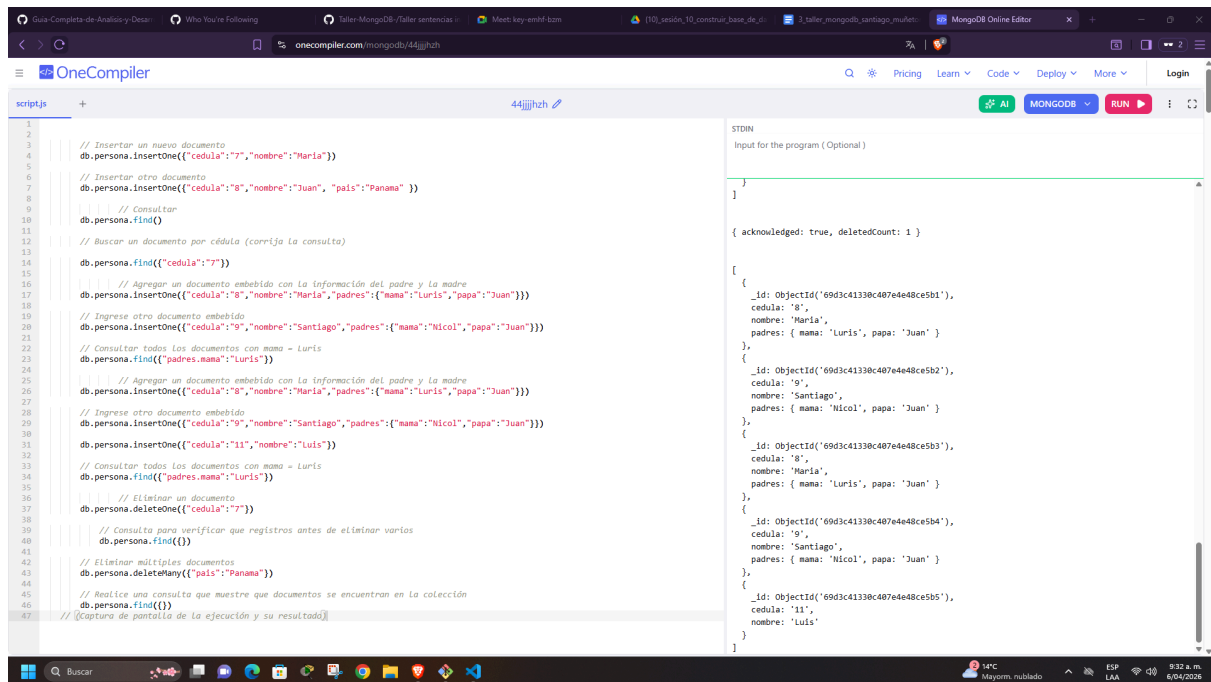
```
db.persona.deleteOne({"cedula":"7"})
```

10. Eliminar múltiples documentos

```
db.persona.deleteMany({"pais":"Panama"})
```

11. Realice una consulta que muestre que documentos se encuentran en la colección

```
db.persona.find({})
```



```
1 // Insertar un nuevo documento
2 db.persona.insertOne({"cedula":"7","nombre":"Maria"})
3
4 // Insertar otro documento
5 db.persona.insertOne({"cedula":"8","nombre":"Juan", "pais":"Panama" })
6
7 // Consultar
8 db.persona.find()
9
10 // Buscar un documento por cédula (corrige la consulta)
11 db.persona.find({"cedula":"7"})
12
13 // Agregar un documento embebido con la información del padre y la madre
14 db.persona.insertOne({"cedula":"8","nombre":"Maria","padres":{"mama":"Luis","papa":"Juan"}})
15
16 // Ingrese otro documento embebido
17 db.persona.insertOne({"cedula":"9","nombre":"Santiago","padres":{"mama":"Nicol","papa":"Juan"}})
18
19 // Consultar todos los documentos con mama = Luis
20 db.persona.find({"padres.mama":"Luis"})
21
22 // Agregar un documento embebido con la información del padre y la madre
23 db.persona.insertOne({"cedula":"9","nombre":"Santiago","padres":{"mama":"Nicol","papa":"Juan"}})
24
25 // Ingrese otro documento embebido
26 db.persona.insertOne({"cedula":"11","nombre":"Luis"})
27
28 // Consultar todos los documentos con mama = Luis
29 db.persona.find({"padres.mama":"Luis"})
30
31 // Eliminar un documento
32 db.persona.deleteOne({"cedula":"7"})
33
34 // Consulta para verificar que registros antes de eliminar varios
35 db.persona.find({})
36
37 // Eliminar múltiples documentos
38 db.persona.deleteMany({"pais":"Panama"})
39
40 // Realice una consulta que muestre que documentos se encuentran en la colección
41 db.persona.find({})
42
43 // Captura de pantalla de la ejecución y su resultado
```

```
STON
Input for the program (Optional)
}
}
{ acknowledged: true, deletedCount: 1 }
[
  {
    _id: ObjectId('69d3c41330c407e4e48ce5b1'),
    cedula: '8',
    nombre: 'Maria',
    padres: { mama: 'Luis', papa: 'Juan' },
  },
  {
    _id: ObjectId('69d3c41330c407e4e48ce5b2'),
    cedula: '9',
    nombre: 'Santiago',
    padres: { mama: 'Nicol', papa: 'Juan' },
  },
  {
    _id: ObjectId('69d3c41330c407e4e48ce5b3'),
    cedula: '9',
    nombre: 'Santiago',
    padres: { mama: 'Nicol', papa: 'Juan' },
  },
  {
    _id: ObjectId('69d3c41330c407e4e48ce5b4'),
    cedula: '9',
    nombre: 'Maria',
    padres: { mama: 'Luis', papa: 'Juan' },
  },
  {
    _id: ObjectId('69d3c41330c407e4e48ce5b5'),
    cedula: '11',
    nombre: 'Luis'
  }
]
```

Conteste: ¿El deleteMany que borro o no borro?, justifique su respuesta.

Elimina únicamente los documentos que contenía el campo "pais":"Panama" y nada más.

Actualizar documentos

12. Crear un documento

```
db.persona.insertOne({"cedula":"6","nombre":"Diana","edad":25})
```

13. Actualizar el nombre

```
db.persona.updateOne({"cedula":"6"},{$set:{"nombre":"Marlenys"}})
```

14. Consulte la colección para observar los cambios

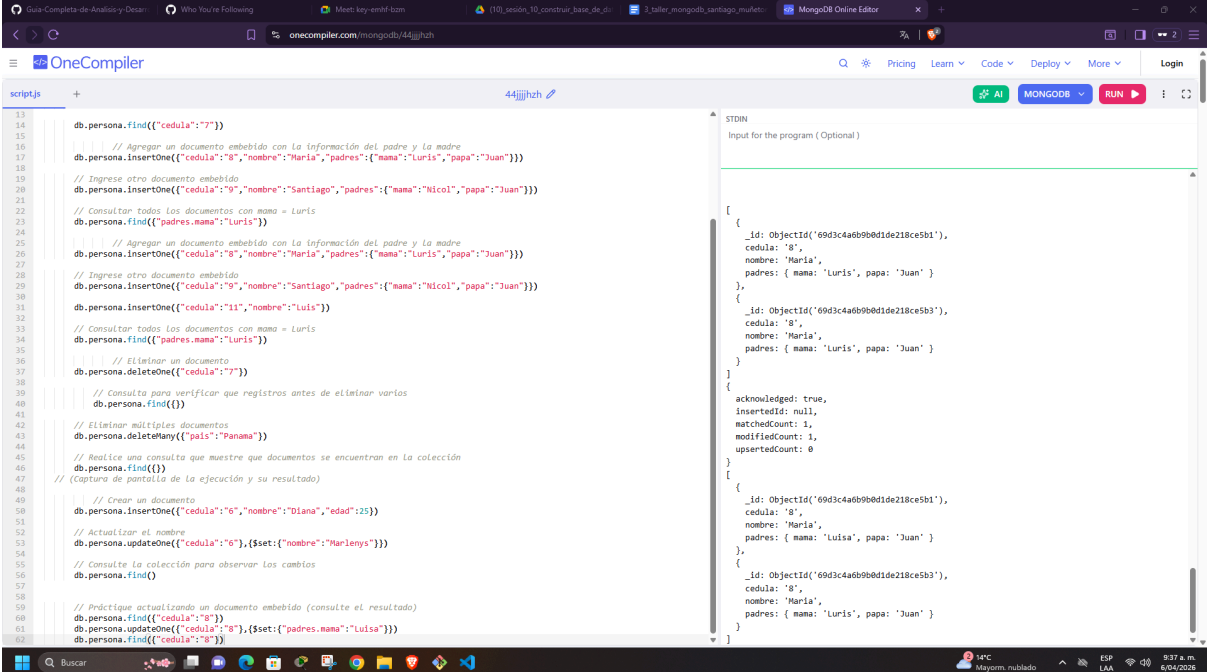
```
db.persona.find()
```

15. Práctique actualizando un documento embebido (consulte el resultado)

```
db.persona.find({"cedula":"8"})

db.persona.updateOne({"cedula":"8"},{$set:{"padres.mama":"Luisa"}})

db.persona.find({"cedula":"8"})
```



```
db.persona.find({"cedula":"7"})

// Agregar un documento embebido con la información del padre y la madre
db.persona.insertOne({"cedula":"8","nombre":"Maria","padres":{"mama":"Luisa","papa":"Juan"}})

// Ingrese otro documento embebido
db.persona.insertOne({"cedula":"9","nombre":"Santiago","padres":{"mama":"Nicol","papa":"Juan"}})

// Consultar todos los documentos con mama = Luis
db.persona.find({"padres.mama":"Luisa"})

// Agregar un documento embebido con la información del padre y la madre
db.persona.insertOne({"cedula":"8","nombre":"Maria","padres":{"mama":"Luisa","papa":"Juan"}})

// Ingrese otro documento embebido
db.persona.insertOne({"cedula":"9","nombre":"Santiago","padres":{"mama":"Nicol","papa":"Juan"}})

db.persona.insertOne({"cedula":"11","nombre":"Luis"})

// Consultar todos los documentos con mama = Luis
db.persona.find({"padres.mama":"Luisa"})

// Eliminar un documento
db.persona.deleteOne({"cedula":"7"})

// Consulta para verificar que registros antes de eliminar varios
db.persona.find({})

// Eliminar multiples documentos
db.persona.deleteMany({"pais":"Panama"})

// Realice una consulta que muestre que documentos se encuentran en la colección
db.persona.find({})

// Captura de pantalla de la ejecución y su resultado

// Crear un documento
db.persona.insertOne({"cedula":"6","nombre":"Diana","edad":25})

// Actualizar el nombre
db.persona.updateOne({"cedula":"6"},{$set:{"nombre":"Marienys"}})

// Consulte la colección para observar los cambios
db.persona.find({})

// Práctique actualizando un documento embebido (consulte el resultado)
db.persona.find({"cedula":"8"})
db.persona.updateOne({"cedula":"8"},{$set:{"padres.mama":"Luisa"}})
db.persona.find({"cedula":"8"})
```

```
{
  "_id": ObjectId('60d3c4a6b9b0d1de218ce5b1'),
  "cedula": '8',
  "nombre": 'Maria',
  "padres": { "mama": 'Luisa', "papa": 'Juan' }
},
{
  "_id": ObjectId('60d3c4a6b9b0d1de218ce5b3'),
  "cedula": '8',
  "nombre": 'Maria',
  "padres": { "mama": 'Luisa', "papa": 'Juan' }
}
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
{
  "_id": ObjectId('60d3c4a6b9b0d1de218ce5b1'),
  "cedula": '8',
  "nombre": 'Maria',
  "padres": { "mama": 'Luisa', "papa": 'Juan' }
},
{
  "_id": ObjectId('60d3c4a6b9b0d1de218ce5b3'),
  "cedula": '8',
  "nombre": 'Maria',
  "padres": { "mama": 'Luisa', "papa": 'Juan' }
}
```

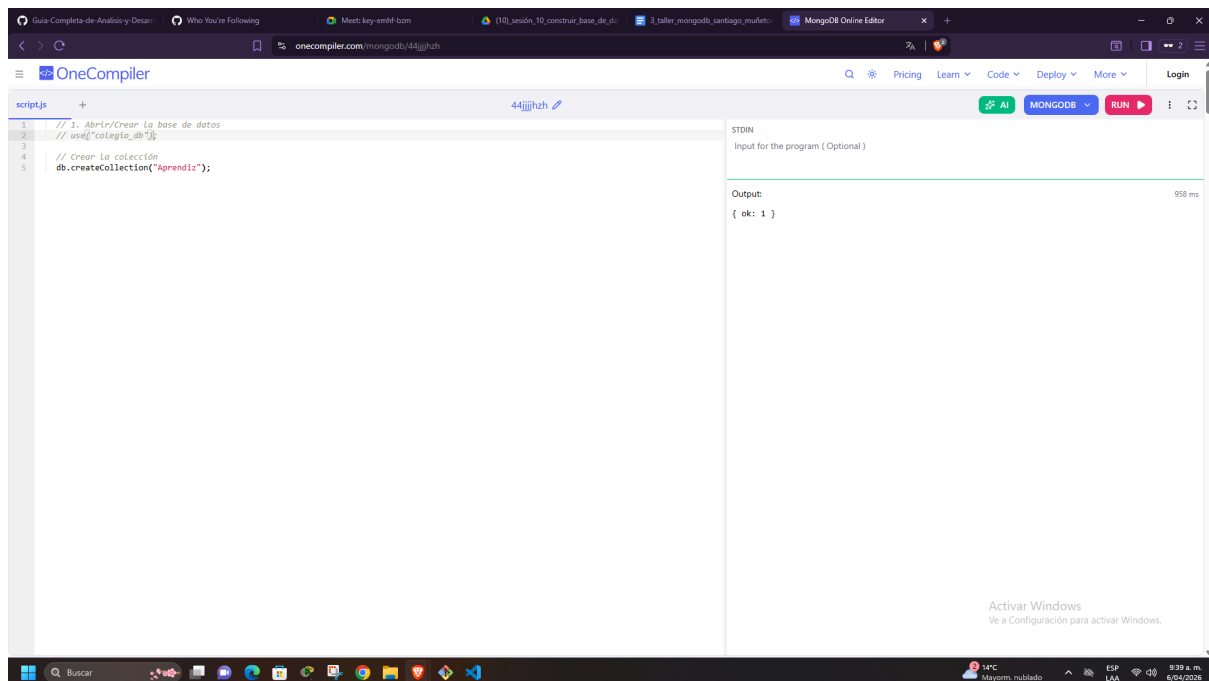
Ahora a continuación la parte 2 del mismo documento.

Cláusulas equivalentes

1. Crear una base de datos que tenga una colección llamada Aprendiz

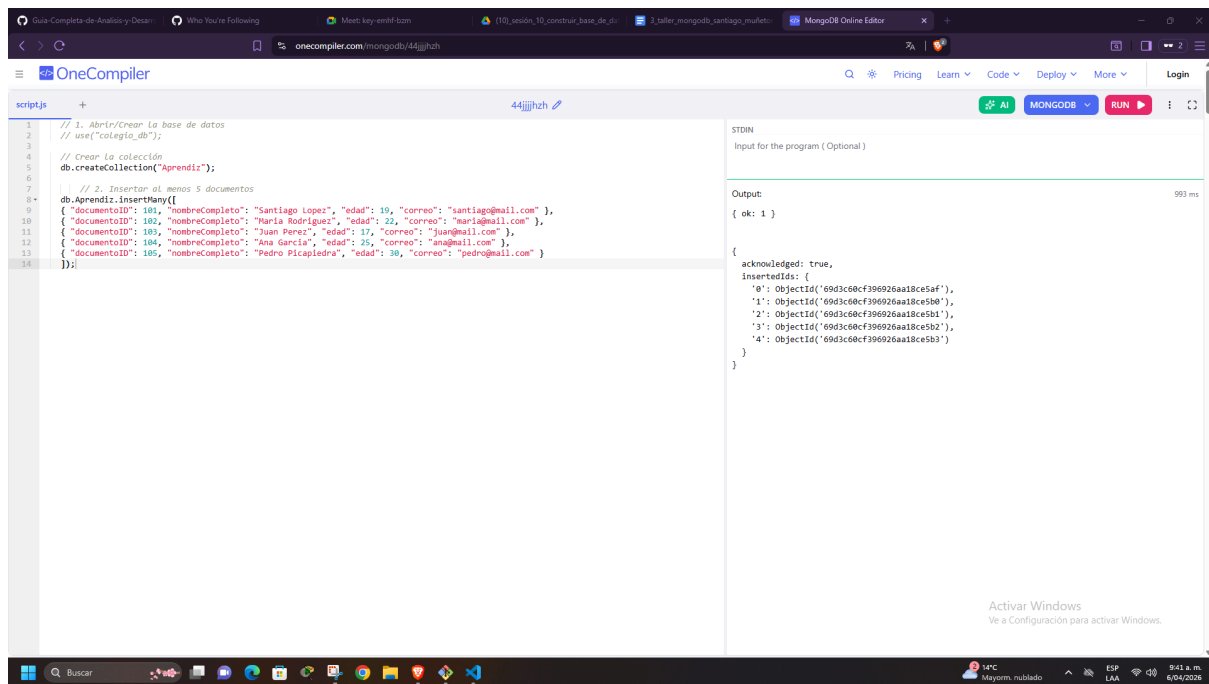
```
// 1. Abrir/Crear la base de datos
use("colegio_db");

// Crear la colección
db.createCollection("Aprendiz");
```



2. Inserte al menos 5 documentos en la colección con los siguientes campos clave: documentoID, nombreCompleto, edad, correo.

```
// 2. Insertar al menos 5 documentos
db.Aprendiz.insertMany([
  { "documentoID": 101, "nombreCompleto": "Santiago Lopez", "edad": 19,
"correo": "santiago@mail.com" },
  { "documentoID": 102, "nombreCompleto": "Maria Rodriguez", "edad": 22,
"correo": "maria@mail.com" },
  { "documentoID": 103, "nombreCompleto": "Juan Perez", "edad": 17,
"correo": "juan@mail.com" },
  { "documentoID": 104, "nombreCompleto": "Ana Garcia", "edad": 25,
"correo": "ana@mail.com" },
  { "documentoID": 105, "nombreCompleto": "Pedro Picapiedra", "edad": 30,
"correo": "pedro@mail.com" }
]);
```



4. Realice las siguientes cláusulas para consultar: \$eq, \$lte, \$gte, \$ne, \$in, \$nin, \$and, \$or, \$nor, \$not

```
// Operadores De Comparación

// $eq: Aprendices que tengan exactamente 22 años
db.Aprendiz.find({ "edad": { $eq: 22 } });

// $lte: Aprendices con edad menor o igual a 19
db.Aprendiz.find({ "edad": { $lte: 19 } });

// $gte: Aprendices con edad mayor o igual a 25
db.Aprendiz.find({ "edad": { $gte: 25 } });

// $ne: Aprendices cuyo documentoID NO sea 101
db.Aprendiz.find({ "documentoID": { $ne: 101 } });

// $in: Aprendices cuyo documentoID esté en la lista [102, 104, 105]
db.Aprendiz.find({ "documentoID": { $in: [102, 104, 105] } });

// $nin: Aprendices cuya edad NO esté en el grupo [17, 30]
db.Aprendiz.find({ "edad": { $nin: [17, 30] } });

// Operadores Lógicos
// $and: Que se llame 'Ana Garcia' Y tenga 25 años
```



```
db.Aprendiz.find({ $and: [ { "nombreCompleto": "Ana Garcia" }, { "edad": 25 } ] });
```

```
// $or: Que tenga 17 años O se llame 'Santiago Lopez'  
db.Aprendiz.find({ $or: [ { "edad": 17 }, { "nombreCompleto": "Santiago Lopez" } ] });
```

```
// $nor: Que NO tenga 19 años Y que tampoco tenga 22  
db.Aprendiz.find({ $nor: [ { "edad": 19 }, { "edad": 22 } ] });
```

```
// $not: Que la edad NO sea mayor a 20 (invierte la condición)  
db.Aprendiz.find({ "edad": { $not: { $gt: 20 } } });
```

The screenshot shows the OneCompiler web IDE interface. On the left, a code editor displays a series of MongoDB queries for a database named 'colegio_db'. The queries include creating a collection, inserting documents, and using various query operators like \$and, \$or, \$nor, and \$not. On the right, the 'STORM' output pane shows the results of the queries, displaying the JSON structure of the documents returned. The interface includes a top navigation bar with links to pricing, learn, code, deploy, and more, as well as a login button. The bottom status bar shows the system time as 9:44 a.m. on 6/04/2025.

```
// 1. Abrir/crear la base de datos  
// use('colegio_db');  
// crear la colección  
db.createCollection('Aprendiz');  
// 2. Insertar al menos 4 documentos  
db.Aprendiz.insertMany([  
  { "documentID": 180, "nombreCompleto": "Santiago Lopez", "edad": 19, "correo": "santiago@mail.com" },  
  { "documentID": 182, "nombreCompleto": "Maria Rodriguez", "edad": 22, "correo": "maria@mail.com" },  
  { "documentID": 183, "nombreCompleto": "Juan Perez", "edad": 17, "correo": "juan@mail.com" },  
  { "documentID": 184, "nombreCompleto": "Ana Garcia", "edad": 25, "correo": "ana@mail.com" },  
  { "documentID": 185, "nombreCompleto": "Pedro Picaledra", "edad": 30, "correo": "pedro@mail.com" }  
]);  
// Operadores de comparación  
// ¿Hay aprendices que tengan exactamente 22 años?  
db.Aprendiz.find({ "edad": { $eq: 22 } });  
// ¿Hay aprendices con edad menor o igual a 19?  
db.Aprendiz.find({ "edad": { $lte: 19 } });  
// ¿Hay aprendices con edad mayor o igual a 25?  
db.Aprendiz.find({ "edad": { $gte: 25 } });  
// ¿Hay aprendices cuyo documentID no sea 182?  
db.Aprendiz.find({ "documentID": { $ne: 182 } });  
// ¿Hay aprendices cuyo documentID esté en la lista [180, 184, 185]?  
db.Aprendiz.find({ "documentID": { $in: [180, 184, 185] } });  
// ¿Hay aprendices cuya edad no esté en el grupo [17, 30]?  
db.Aprendiz.find({ "edad": { $nin: [17, 30] } });  
// Operadores lógicos  
// ¿Hay alguien que se llame 'Ana Garcia' y tenga 25 años?  
db.Aprendiz.find({ $and: [ { "nombreCompleto": "Ana Garcia" }, { "edad": 25 } ] });  
// ¿Hay alguien que tenga 17 años o se llame 'Santiago Lopez'?  
db.Aprendiz.find({ $or: [ { "edad": 17 }, { "nombreCompleto": "Santiago Lopez" } ] });  
// ¿Hay alguien que NO tenga 19 años y que tampoco tenga 22?  
db.Aprendiz.find({ $nor: [ { "edad": 19 }, { "edad": 22 } ] });  
// ¿Hay alguien que la edad NO sea mayor a 20 (invierte la condición)?  
db.Aprendiz.find({ "edad": { $not: { $gt: 20 } } });
```

STORM
Input for the program (Optional)

```
{  
  "_id": "60d3c6b7e091e11a3ce5b3",  
  "documentID": 180,  
  "nombreCompleto": "Pedro Picaledra",  
  "edad": 30,  
  "correo": "pedro@mail.com"  
},  
{  
  "_id": "60d3c6b7e091e11a3ce5b0",  
  "documentID": 182,  
  "nombreCompleto": "Maria Rodriguez",  
  "edad": 22,  
  "correo": "maria@mail.com"  
},  
{  
  "_id": "60d3c6b7e091e11a3ce5b2",  
  "documentID": 184,  
  "nombreCompleto": "Ana Garcia",  
  "edad": 25,  
  "correo": "ana@mail.com"  
},  
{  
  "_id": "60d3c6b7e091e11a3ce5b4",  
  "documentID": 180,  
  "nombreCompleto": "Santiago Lopez",  
  "edad": 19,  
  "correo": "santiago@mail.com"  
},  
{  
  "_id": "60d3c6b7e091e11a3ce5b0",  
  "documentID": 182,  
  "nombreCompleto": "Maria Rodriguez",  
  "edad": 22,  
  "correo": "maria@mail.com"  
}
```

Activar Windows
Ve a Configuración para activar Windows.